DCM Analysis Report

Deadly Cocktail Model — Full Statistical Analysis and Visualisations

Contents

Data Loading . . . 2
Analysis A: Cross-Category DCM Prevalence . . . 2
A1: Prevalence with Wilson CIs . . . 2
A2: Chi-Square Tests of Independence . . . 4
A3: Logistic Regression with Cluster-Robust Standard Errors . . . 4
Analysis B: Feature Co-occurrence . . . 6
B1: Feature Count Distribution . . . 6
B2: Pairwise Co-occurrence . . . 7
B3: Full Cocktail Test . . . 10
Analysis C: Channel-Level Variance . . . 11
C1: Per-Channel Detail . . . 11
C2: Summary Statistics of Per-Channel Prevalence . . . 11
Channel-Level Prevalence Distribution . . . 12
Analysis D: Topic x DCM Integration . . . 13
Data Preparation . . . 13
D1: Topic-Feature Proportions . . . 14
D2: Topic Toxicity Ranking . . . 14
D3: Topic Feature Profiles . . . 15
D4: Logistic Regression — Topic as Predictor of DCM Features . . . 16
Topic-Feature Heatmap . . . 17
Topic Toxicity Ranking (Top 20) . . . 20
Analysis E: Theme Prevalence Across Channel Types . . . 21
Data Preparation . . . 21
E1: Theme Prevalence Proportions . . . 22
E2: Chi-Square Test (Theme x Category) . . . 23
E3: Standardized Residuals . . . 24
Analysis F: DCM-to-Topic (Reverse Direction) . . . 25
F1: Topic Distribution for DCM+ vs DCM- Messages . . . 25
F2: Theme Distribution of DCM+ Messages . . . 26
F3: Log-Odds Topic Enrichment . . . 27
Analysis G: Shared vs. Movement-Unique Theme DCM Check . . . 29
G1: Theme Overlap Classification . . . 29
G2: Topic-Level Classification and DCM Comparison . . . 29
G3: Statistical Test . . . 31
G4: Figure — Composite DCM by Overlap Status . . . 32
Temporal DCM Prevalence . . . 33
Transfer Entropy . . . 35
Directed Temporal Dependencies . . . 35

Data Loading

```
df <- read_csv(DATA_PATH, col_types = cols(.default = "c")) %>%
  select(category, channel, all_of(FEATURES)) %>%
  mutate(across(all_of(FEATURES), as.integer))
```

The corpus contains **742,264** messages across **43** channels in **3** movement categories (764, terrorgram, white-supremacist).

Analysis A: Cross-Category DCM Prevalence

Analysis A compares the prevalence of each DCM feature (Identity Fusion, Violence Condoning, Existential Threat, Outgroup Othering) across the three movement categories. Descriptive rates are reported with Wilson 95% confidence intervals, followed by chi-square tests of independence and logistic regression with cluster-robust standard errors (clustered by channel) to account for the non-independence of messages within channels.

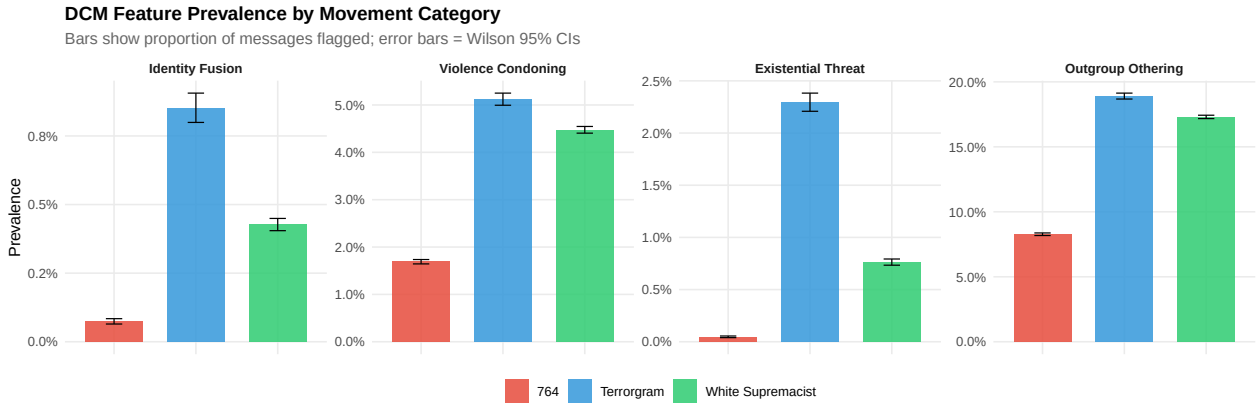
A1: Prevalence with Wilson CIs

```
wilson_ci <- function(k, n, alpha = 0.05) {
  if (n == 0) return(tibble(prev = NA, ci_lo = NA, ci_hi = NA))
  p <- k / n
  z <- qnorm(1 - alpha / 2)
  denom <- 1 + z^2 / n
  centre <- (p + z^2 / (2 * n)) / denom
  margin <- z * sqrt(p * (1 - p) / n + z^2 / (4 * n^2)) / denom
  tibble(prev = p, ci_lo = max(0, centre - margin), ci_hi = min(1, centre + margin))
}

prevalence <- map_dfr(FEATURES, function(feat) {
  df %>%
    filter(!is.na(.data[[feat]])) %>%
    group_by(category) %>%
    summarise(n = n(), k = sum(.data[[feat]]), .groups = "drop") %>%
    rowwise() %>%
    mutate(wilson_ci(k, n)) %>%
    ungroup() %>%
    mutate(feature = FEATURE_LABELS[feat])
}) %>%
  select(feature, category, n, k, prev, ci_lo, ci_hi)

write_csv(prevalence, file.path(OUTPUT_DIR, "A_prevalence.csv"))

prevalence %>%
  mutate(across(c(prev, ci_lo, ci_hi), ~ percent(., accuracy = 0.1))) %>%
  kable(col.names = c("Feature", "Category", "N", "Flagged", "Prevalence", "CI Low", "CI High"),
        caption = "DCM feature prevalence by movement category (Wilson 95% CIs)")
```

Table 1: DCM feature prevalence by movement category (Wilson 95% CIs)

Feature	Category	N	Flagged	Prevalence	CI Low	CI High
Identity Fusion	764	299708	221	0.1%	0.1%	0.1%
Identity Fusion	terrorgram	113624	967	0.9%	0.8%	0.9%
Identity Fusion	white-supremacist	328701	1402	0.4%	0.4%	0.4%
Violence Condoning	764	299793	5065	1.7%	1.6%	1.7%
Violence Condoning	terrorgram	113654	5820	5.1%	5.0%	5.3%
Violence Condoning	white-supremacist	328737	14711	4.5%	4.4%	4.5%
Existential Threat	764	299775	140	0.0%	0.0%	0.1%
Existential Threat	terrorgram	113647	2607	2.3%	2.2%	2.4%
Existential Threat	white-supremacist	328715	2507	0.8%	0.7%	0.8%
Outgroup Othering	764	299797	24804	8.3%	8.2%	8.4%
Outgroup Othering	terrorgram	113661	21489	18.9%	18.7%	19.1%
Outgroup Othering	white-supremacist	328737	56884	17.3%	17.2%	17.4%

```r
prev_plot <- prevalence %>%
  mutate(
    feature = factor(feature, levels = FEATURE_ORDER),
    category = factor(category, levels = names(CATEGORY_LABELS))
  )

p_prev <- ggplot(prev_plot, aes(x = category, y = prev, fill = category)) +
  geom_col(width = 0.7, alpha = 0.85) +
  geom_errorbar(aes(ymin = ci_lo, ymax = ci_hi), width = 0.2, linewidth = 0.4) +
  facet_wrap(~feature, scales = "free_y", nrow = 1) +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_x_discrete(labels = CATEGORY_LABELS) +
  scale_y_continuous(labels = percent_format(accuracy = 0.1)) +
  labs(
    title = "DCM Feature Prevalence by Movement Category",
    subtitle = "Bars show proportion of messages flagged; error bars = Wilson 95% CIs",
    x = NULL, y = "Prevalence"
  ) +
  theme_pub() +
  theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())

p_prev
```



```r
ggsave(file.path(FIGURES_DIR, "prevalence.pdf"), p_prev, width = 12, height = 4)
```

A2: Chi-Square Tests of Independence

```r
chisq_results <- map_dfr(FEATURES, function(feat) {
  sub <- df %>% filter(!is.na(.data[[feat]]))
  ct <- table(sub$category, sub[[feat]])
  test <- chisq.test(ct)
  n <- sum(ct)
  cramers_v <- sqrt(test$statistic / (n * (min(dim(ct)) - 1)))
  tibble(
    feature   = FEATURE_LABELS[feat],
    chi_sq    = as.numeric(test$statistic),
    df        = test$parameter,
    p_value   = test$p.value,
    cramers_v = as.numeric(cramers_v)
  )
})

write_csv(chisq_results, file.path(OUTPUT_DIR, "A_chisquare.csv"))

chisq_results %>%
  mutate(
    chi_sq    = round(chi_sq, 1),
    p_value   = formatC(p_value, format = "e", digits = 2),
    cramers_v = round(cramers_v, 4)
  ) %>%
  kable(col.names = c("Feature", "Chi-sq", "df", "p-value", "Cramer's V"),
        caption = "Chi-square tests of feature x category independence")
```

Table 2: Chi-square tests of feature x category independence

Feature	Chi-sq	df	p-value	Cramer's V
Identity Fusion	1533.1	2	0.00e+00	0.0455
Violence Condoning	4780.5	2	0.00e+00	0.0803
Existential Threat	5945.4	2	0.00e+00	0.0895
Outgroup Othering	13491.3	2	0.00e+00	0.1348

A3: Logistic Regression with Cluster-Robust Standard Errors

Standard GLM with sandwich/vcovCL cluster-robust (Huber-White) standard errors, clustered by channel. This is equivalent to GEE with independence working correlation and robust variance estimation. Reference category: 764.

```r
gee_results <- map_dfr(FEATURES, function(feat) {
  sub <- df %>%
    filter(!is.na(.data[[feat]])) %>%
    mutate(
      y = .data[[feat]],
      category = relevel(factor(category), ref = REF_CAT),
      channel_f = factor(channel)
    )
```

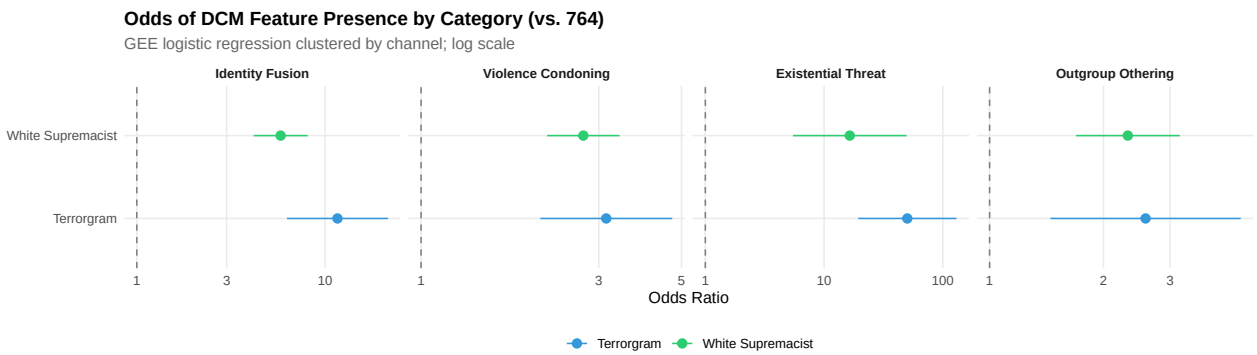
```r
  fit <- glm(y ~ category, data = sub, family = binomial(link = "logit"))
  robust <- coeftest(fit, vcov = vcovCL(fit, cluster = sub$channel_f))

  as_tibble(robust[, , drop = FALSE], rownames = "term") %>%
    rename(estimate = Estimate, std.error = `Std. Error`,
           statistic = `z value`, p.value = `Pr(>|z|)`) %>%
    filter(term != "(Intercept)") %>%
    mutate(
      feature    = FEATURE_LABELS[feat],
      comparison = str_remove(term, "^category"),
      odds_ratio = exp(estimate),
      ci_lo      = exp(estimate - 1.96 * std.error),
      ci_hi      = exp(estimate + 1.96 * std.error)
    ) %>%
    select(feature, comparison, odds_ratio, ci_lo, ci_hi, p_value = p.value)
})

write_csv(gee_results, file.path(OUTPUT_DIR, "A_gee.csv"))

gee_results %>%
  mutate(
    across(c(odds_ratio, ci_lo, ci_hi), ~ round(., 3)),
    p_value = formatC(p_value, format = "e", digits = 2)
  ) %>%
  kable(col.names = c("Feature", "Comparison (vs. 764)", "OR", "CI Low", "CI High", "p-value"),
        caption = "Odds ratios from cluster-robust logistic regression (reference = 764)")
```

Table 3: Odds ratios from cluster-robust logistic regression (reference = 764)

Feature	Comparison (vs. 764)	OR	CI Low	CI High	p-value
Identity Fusion	terrorgram	11.632	6.274	21.566	6.70e-15
Identity Fusion	white-supremacist	5.805	4.173	8.074	1.53e-25
Violence Condoning	terrorgram	3.141	2.089	4.722	3.79e-08
Violence Condoning	white-supremacist	2.726	2.179	3.410	1.71e-18
Existential Threat	terrorgram	50.249	19.374	130.330	7.93e-16
Existential Threat	white-supremacist	16.448	5.474	49.428	6.09e-07
Outgroup Othering	terrorgram	2.585	1.448	4.613	1.31e-03
Outgroup Othering	white-supremacist	2.320	1.691	3.183	1.83e-07

```r
gee_plot <- gee_results %>%
  mutate(
    feature = factor(feature, levels = FEATURE_ORDER),
    comparison = str_replace_all(comparison, c(
      "terrorgram"       = "Terrorgram",
      "white-supremacist" = "White Supremacist"
    ))
  )

p_gee <- ggplot(gee_plot, aes(x = odds_ratio, y = comparison, colour = comparison)) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey50") +
  geom_pointrange(aes(xmin = ci_lo, xmax = ci_hi), linewidth = 0.5, size = 0.5) +
```

```r
  facet_wrap(~feature, scales = "free_x", nrow = 1) +
  scale_colour_manual(
    values = c("Terrorgram" = "#3498db", "White Supremacist" = "#2ecc71"),
    name = NULL
  ) +
  scale_x_log10() +
  labs(
    title = "Odds of DCM Feature Presence by Category (vs. 764)",
    subtitle = "GEE logistic regression clustered by channel; log scale",
    x = "Odds Ratio", y = NULL
  ) +
  theme_pub()

p_gee
```




```r
ggsave(file.path(FIGURES_DIR, "gee_forest.pdf"), p_gee, width = 12, height = 3.5)
```

Analysis B: Feature Co-occurrence

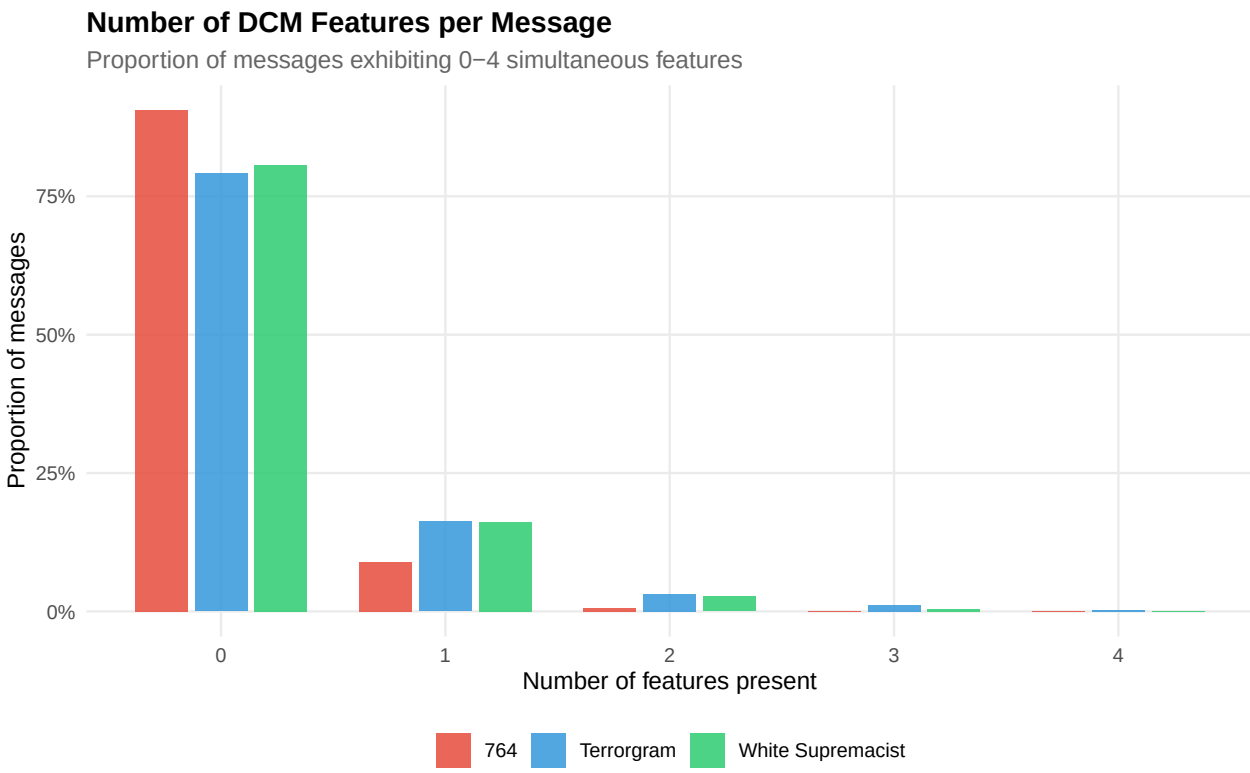
Analysis B examines whether DCM features co-occur within the same messages more than expected by chance — the "Deadly Cocktail" hypothesis. It reports the distribution of feature counts per message (0–4 simultaneous features), pairwise phi coefficients for each feature pair, and a binomial test of whether the "full cocktail" (all four features present) exceeds the rate expected under independence.

B1: Feature Count Distribution

```r
df_complete <- df %>% filter(if_all(all_of(FEATURES), ~ !is.na(.)))
df_complete <- df_complete %>%
  mutate(n_features = rowSums(across(all_of(FEATURES))))

count_dist <- df_complete %>%
  group_by(category, n_features) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(category) %>%
  mutate(prop = n / sum(n)) %>%
  ungroup()

write_csv(count_dist, file.path(OUTPUT_DIR, "B_feature_count_distribution.csv"))

count_overall <- df_complete %>%
```

```r
  count(n_features) %>%
  mutate(prop = n / sum(n))

write_csv(count_overall, file.path(OUTPUT_DIR, "B_feature_count_overall.csv"))
```

```r
counts_plot <- count_dist %>%
  mutate(category = factor(category, levels = names(CATEGORY_LABELS)))

p_counts <- ggplot(counts_plot, aes(x = factor(n_features), y = prop, fill = category)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7, alpha = 0.85) +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Number of DCM Features per Message",
    subtitle = "Proportion of messages exhibiting 0\u20134 simultaneous features",
    x = "Number of features present", y = "Proportion of messages"
  ) +
  theme_pub()

p_counts
```




```r
ggsave(file.path(FIGURES_DIR, "feature_counts.pdf"), p_counts, width = 8, height = 5)
```

B2: Pairwise Co-occurrence

```r
feat_pairs <- combn(FEATURES, 2, simplify = FALSE)

pairwise <- map_dfr(feat_pairs, function(pair) {
  a <- pair[1]; b <- pair[2]
```

```r
  va <- df_complete[[a]]; vb <- df_complete[[b]]
  n <- length(va)
  obs <- sum(va == 1 & vb == 1)
  pa <- mean(va); pb <- mean(vb)
  exp_val <- pa * pb * n
  ct <- table(factor(va, 0:1), factor(vb, 0:1))
  chi <- chisq.test(ct, correct = FALSE)
  phi <- sqrt(chi$statistic / n) * sign(obs - exp_val)
  a11 <- as.double(ct[2, 2]); a10 <- as.double(ct[2, 1])
  a01 <- as.double(ct[1, 2]); a00 <- as.double(ct[1, 1])
  or <- (a11 * a00) / (a10 * a01)
  tibble(
    feature_a    = FEATURE_LABELS[a],
    feature_b    = FEATURE_LABELS[b],
    observed     = obs,
    expected     = round(exp_val, 1),
    obs_exp_ratio = round(obs / exp_val, 2),
    phi          = round(as.numeric(phi), 4),
    odds_ratio   = round(as.numeric(or), 2),
    chi_sq       = round(as.numeric(chi$statistic), 1),
    p_value      = chi$p.value
  )
})

write_csv(pairwise, file.path(OUTPUT_DIR, "B_pairwise_cooccurrence.csv"))

pairwise %>%
  mutate(p_value = formatC(p_value, format = "e", digits = 2)) %>%
  kable(caption = "Pairwise co-occurrence: observed vs. expected, phi, and odds ratios (overall)")
```

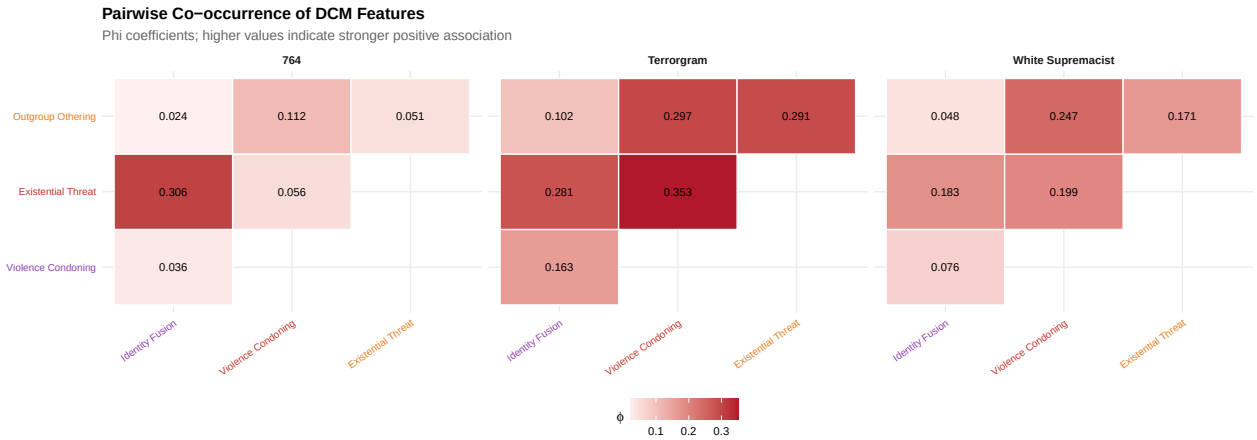
Table 4: Pairwise co-occurrence: observed vs. expected, phi, and odds ratios (overall)

feature_a	feature_b	observed	expected	obs_exp_ratio	phi	odds_ratio	chi_sq	p_value
Identity Fusion	Violence Condoning	862	89.1	9.67	0.0969	14.46	6963.9	0.00e+00
Identity Fusion	Existential Threat	865	18.3	47.32	0.2311	84.36	39640.4	0.00e+00
Identity Fusion	Outgroup Othering	1303	359.5	3.62	0.0624	6.36	2885.6	0.00e+00
Violence Condoning	Existential Threat	2789	180.8	15.43	0.2300	35.59	39253.7	0.00e+00
Violence Condoning	Outgroup Othering	14524	3555.7	4.08	0.2342	9.31	40700.0	0.00e+00
Existential Threat	Outgroup Othering	4812	729.1	6.60	0.1899	72.31	26746.0	0.00e+00

```r
pairwise_by_cat <- map_dfr(feat_pairs, function(pair) {
  a <- pair[1]; b <- pair[2]
  map_dfr(unique(df_complete$category), function(cat) {
    sub <- df_complete %>% filter(category == cat)
    va <- sub[[a]]; vb <- sub[[b]]
```

```r
    n <- length(va)
    obs <- sum(va == 1 & vb == 1)
    exp_val <- mean(va) * mean(vb) * n
    ct <- table(factor(va, 0:1), factor(vb, 0:1))
    chi <- suppressWarnings(chisq.test(ct, correct = FALSE))
    phi <- sqrt(chi$statistic / n) * sign(obs - exp_val)
    tibble(
      feature_a    = FEATURE_LABELS[a],
      feature_b    = FEATURE_LABELS[b],
      category     = cat,
      observed     = obs,
      expected     = round(exp_val, 1),
      obs_exp_ratio = round(obs / max(exp_val, 1), 2),
      phi          = round(as.numeric(phi), 4),
      p_value      = chi$p.value
    )
  })
})

write_csv(pairwise_by_cat, file.path(OUTPUT_DIR, "B_pairwise_by_category.csv"))
```

```r
pairs_plot <- pairwise_by_cat %>%
  mutate(
    feature_a = factor(feature_a, levels = FEATURE_ORDER),
    feature_b = factor(feature_b, levels = FEATURE_ORDER),
    category = factor(category, levels = names(CATEGORY_LABELS))
  )

p_cooc <- ggplot(pairs_plot, aes(x = feature_a, y = feature_b, fill = phi)) +
  geom_tile(colour = "white", linewidth = 0.5) +
  geom_text(aes(label = sprintf("%.3f", phi)), size = 3) +
  facet_wrap(~category, labeller = labeller(category = CATEGORY_LABELS)) +
  scale_fill_gradient2(low = "#2166ac", mid = "white", high = "#b2182b",
                       midpoint = 0, name = expression(phi)) +
  labs(
    title = "Pairwise Co-occurrence of DCM Features",
    subtitle = "Phi coefficients; higher values indicate stronger positive association",
    x = NULL, y = NULL
  ) +
  theme_pub() +
  theme(
    axis.text.x = element_text(angle = 35, hjust = 1, size = 8,
                               colour = FEATURE_COLORS[levels(pairs_plot$feature_a)]),
    axis.text.y = element_text(size = 8,
                               colour = FEATURE_COLORS[levels(pairs_plot$feature_b)])
  )

p_cooc
```




```
ggsave(file.path(FIGURES_DIR, "cooccurrence_phi.pdf"), p_cooc, width = 14, height = 5)
```

B3: Full Cocktail Test

Under independence, P(all 4 features) = P(IF) x P(VL) x P(TH) x P(OO). A one-sided binomial test assesses whether the observed count of "full cocktail" messages significantly exceeds this expected rate.

```
cocktail <- df_complete %>%
  group_by(category) %>%
  summarise(
    n = n(),
    all_four = sum(n_features == 4),
    p_if = mean(identityfusion_bin),
    p_vl = mean(violentlang_bin),
    p_th = mean(threat_bin),
    p_oo = mean(outgroup_othering_bin),
    .groups = "drop"
  ) %>%
  mutate(
    p_indep  = p_if * p_vl * p_th * p_oo,
    expected = p_indep * n,
    ratio    = round(all_four / pmax(expected, 0.01), 1),
    binom_p  = pbinom(all_four - 1, n, p_indep, lower.tail = FALSE)
  )

write_csv(
  cocktail %>% select(category, n, all_four, expected, ratio, binom_p),
  file.path(OUTPUT_DIR, "B_full_cocktail_test.csv")
)

cocktail %>%
  select(category, n, all_four, expected, ratio, binom_p) %>%
  mutate(
    expected = round(expected, 1),
    binom_p  = formatC(binom_p, format = "e", digits = 2)
  ) %>%
  kable(col.names = c("Category", "N", "Observed (all 4)", "Expected", "Obs/Exp Ratio", "Binomial p"),
        caption = "Full cocktail test: observed vs. expected frequency of all 4 features co-occurring")
```

Table 5: Full cocktail test: observed vs. expected frequency of all 4 features co-occurring

Category	N	Observed (all 4)	Expected	Obs/Exp Ratio	Binomial p
764	299667	16	0.0	1600.0	9.51e-76
terrorgram	113602	301	0.2	1403.9	0.00e+00
white-supremacist	328641	205	0.1	2481.4	0.00e+00

Analysis C: Channel-Level Variance

Analysis C provides a robustness check by disaggregating results to the channel level. It reports per-channel DCM feature prevalence rates and summary statistics (mean, SD, quartiles) of per-channel prevalence by category. This demonstrates that the cross-group differences observed in Analysis A are not driven by a few high-volume or outlier channels, and provides a measure of within-category heterogeneity.

C1: Per-Channel Detail

```
channel_detail <- df %>%
  group_by(category, channel) %>%
  summarise(
    n_messages = n(),
    across(all_of(FEATURES), ~ mean(., na.rm = TRUE), .names = "prev_{.col}"),
    .groups = "drop"
  ) %>%
  arrange(category, desc(n_messages))

write_csv(channel_detail, file.path(OUTPUT_DIR, "C_channel_detail.csv"))
```

C2: Summary Statistics of Per-Channel Prevalence

```
channel_dcm_summary <- map_dfr(FEATURES, function(feat) {
  prev_col <- paste0("prev_", feat)
  channel_detail %>%
    group_by(category) %>%
    summarise(
      feature     = FEATURE_LABELS[feat],
      n_channels  = n(),
      mean_prev   = mean(.data[[prev_col]], na.rm = TRUE),
      sd_prev     = sd(.data[[prev_col]], na.rm = TRUE),
      min_prev    = min(.data[[prev_col]], na.rm = TRUE),
      q25_prev    = quantile(.data[[prev_col]], 0.25, na.rm = TRUE),
      median_prev = median(.data[[prev_col]], na.rm = TRUE),
      q75_prev    = quantile(.data[[prev_col]], 0.75, na.rm = TRUE),
      max_prev    = max(.data[[prev_col]], na.rm = TRUE),
      .groups = "drop"
    )
})

write_csv(channel_dcm_summary, file.path(OUTPUT_DIR, "C_channel_dcm_summary.csv"))

channel_dcm_summary %>%
```

```
  mutate(across(c(mean_prev, sd_prev, min_prev, q25_prev, median_prev, q75_prev, max_prev),
                ~ percent(., accuracy = 0.1))) %>%
  kable(col.names = c("Category", "Feature", "Channels", "Mean", "SD", "Min", "Q25", "Median", "Q75", "
        caption = "Summary statistics of per-channel DCM prevalence by category")
```

Table 6: Summary statistics of per-channel DCM prevalence by category

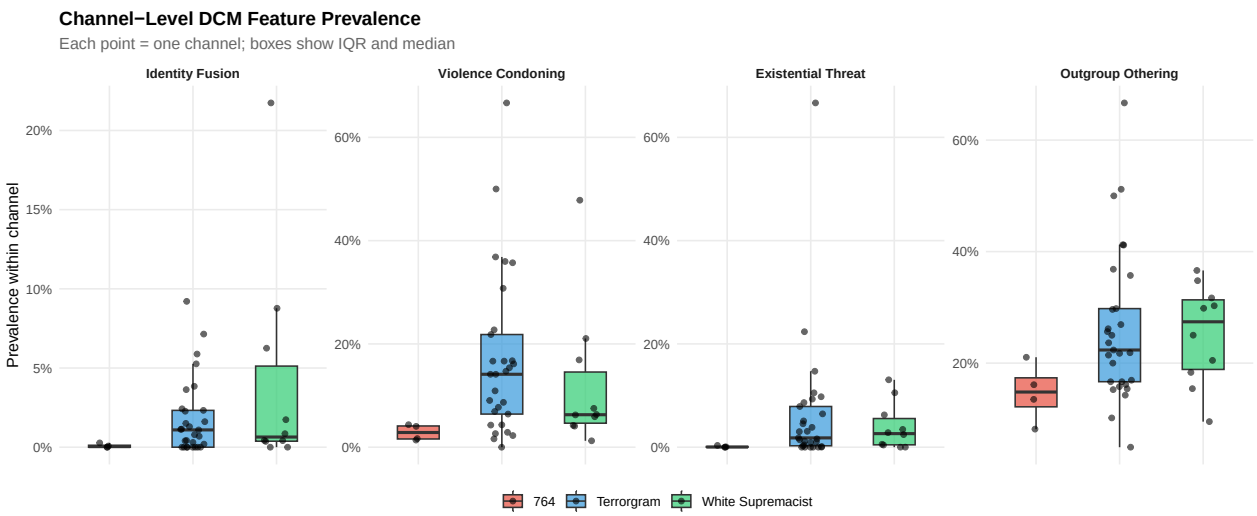
Category	Feature	Channels	Mean	SD	Min	Q25	Median	Q75	Max
764	Identity Fusion	4	0.1%	0.1%	0.0%	0.0%	0.0%	0.1%	0.3%
terrorgram	Identity Fusion	29	1.8%	2.4%	0.0%	0.0%	1.1%	2.3%	9.2%
white-supremacist	Identity Fusion	10	4.1%	6.9%	0.0%	0.4%	0.6%	5.1%	21.7%
764	Violence Condoning	4	2.9%	1.6%	1.4%	1.6%	2.8%	4.1%	4.4%
terrorgram	Violence Condoning	29	17.0%	15.7%	0.0%	6.4%	14.1%	21.8%	66.7%
white-supremacist	Violence Condoning	10	12.1%	13.9%	1.2%	4.6%	6.3%	14.6%	47.8%
764	Existential Threat	4	0.1%	0.2%	0.0%	0.0%	0.0%	0.1%	0.4%
terrorgram	Existential Threat	29	6.4%	12.7%	0.0%	0.3%	1.8%	7.9%	66.7%
white-supremacist	Existential Threat	10	3.9%	4.6%	0.0%	0.5%	2.6%	5.5%	13.0%
764	Outgroup Othering	4	14.7%	5.4%	8.2%	12.2%	14.8%	17.4%	21.1%
terrorgram	Outgroup Othering	29	26.2%	13.6%	4.9%	16.7%	22.4%	29.8%	66.7%
white-supremacist	Outgroup Othering	10	25.2%	9.0%	9.5%	18.9%	27.4%	31.3%	36.6%

Channel-Level Prevalence Distribution

Each point represents one channel's prevalence rate for a given DCM feature. Boxes show the interquartile range and median.

```
channel_long <- channel_detail %>%
  pivot_longer(cols = starts_with("prev_"), names_to = "feature", values_to = "prevalence") %>%
  mutate(
    feature = str_remove(feature, "prev_"),
    feature = recode(feature,
      identityfusion_bin = "Identity Fusion",
      violentlang_bin = "Violence Condoning",
      threat_bin = "Existential Threat",
      outgroup_othering_bin = "Outgroup Othering"
    ),
    feature = factor(feature, levels = FEATURE_ORDER),
    category = factor(category, levels = names(CATEGORY_LABELS))
  )

p_channel <- ggplot(channel_long, aes(x = category, y = prevalence, fill = category)) +
  geom_boxplot(width = 0.5, alpha = 0.7, outlier.shape = NA) +
```

```
  geom_jitter(width = 0.15, size = 1.5, alpha = 0.6) +
  facet_wrap(~feature, scales = "free_y", nrow = 1) +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_x_discrete(labels = CATEGORY_LABELS) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Channel-Level DCM Feature Prevalence",
    subtitle = "Each point = one channel; boxes show IQR and median",
    x = NULL, y = "Prevalence within channel"
  ) +
  theme_pub() +
  theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())

p_channel
```




```
ggsave(file.path(FIGURES_DIR, "channel_variance.pdf"), p_channel, width = 12, height = 5)
```

Analysis D: Topic x DCM Integration

Analysis D bridges the topic modelling results (Part 1) with the DCM annotation (Part 2) by asking: which topics are most associated with which DCM features? For each of the 19 topics per category, it computes the proportion of messages flagged for each DCM feature. Topics are then ranked by a composite DCM score (mean of the four feature proportions) and classified into profiles (e.g., "Full cocktail," "Othering + Violence"). A logistic regression tests which topics are significant predictors of each DCM feature within each category.

Data Preparation

```
TOPICS_PATH <- file.path(BASE_DIR, "topic_modelling/step1_labels/telegram_topics_message_topics.csv")
LABELS_PATH <- file.path(BASE_DIR, "topic_modelling/step1_labels/topic_labels.csv")

topics <- read_csv(TOPICS_PATH, col_types = cols(.default = "c")) %>%
  rename(channel_name = channel) %>%
  mutate(topic = as.integer(as.numeric(topic)))
```

```r
df_full <- read_csv(DATA_PATH, col_types = cols(.default = "c")) %>%
  select(category, channel, channel_name, text, all_of(FEATURES)) %>%
  mutate(across(all_of(FEATURES), as.integer))

df_full <- df_full %>%
  mutate(text_lower = tolower(text)) %>%
  left_join(
    topics %>%
      mutate(text_lower = tolower(text)) %>%
      select(channel_name, category, text_lower, topic),
    by = c("channel_name", "category", "text_lower")
  ) %>%
  select(-text, -text_lower)

topic_labels <- read_csv(LABELS_PATH, col_types = cols(.default = "c")) %>%
  select(category, topic, label) %>%
  mutate(topic = as.integer(as.numeric(topic)))

df_topics <- df_full %>%
  filter(!is.na(topic)) %>%
  left_join(topic_labels, by = c("category", "topic"))
```

Topic merge matched **720,843** of **742,264** messages (97.1%).

D1: Topic-Feature Proportions

```r
topic_props <- df_topics %>%
  group_by(category, topic, label) %>%
  summarise(
    n = n(),
    identity_fusion    = mean(identityfusion_bin, na.rm = TRUE),
    violence_condoning = mean(violentlang_bin, na.rm = TRUE),
    existential_threat = mean(threat_bin, na.rm = TRUE),
    outgroup_othering  = mean(outgroup_othering_bin, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    composite_dcm = (identity_fusion + violence_condoning +
                     existential_threat + outgroup_othering) / 4
  ) %>%
  arrange(category, desc(composite_dcm))

write_csv(topic_props, file.path(OUTPUT_DIR, "D_topic_feature_proportions.csv"))
```

D2: Topic Toxicity Ranking

```r
toxic_ranking <- topic_props %>%
  filter(topic >= 0) %>%
  arrange(desc(composite_dcm)) %>%
  mutate(rank = row_number())

write_csv(toxic_ranking, file.path(OUTPUT_DIR, "D_topic_toxicity_ranking.csv"))
```

```r
toxic_ranking %>%
  filter(rank <= 10) %>%
  mutate(across(c(identity_fusion, violence_condoning, existential_threat,
                  outgroup_othering, composite_dcm), ~ percent(., accuracy = 0.1))) %>%
  select(rank, category, label, n, composite_dcm) %>%
  kable(col.names = c("Rank", "Category", "Topic", "N", "Composite DCM"),
        caption = "Top 10 topics by composite DCM score")
```

Table 7: Top 10 topics by composite DCM score

Rank	Category	Topic	N	Composite DCM
1	terrorgram	Social Media/Antisemitism	553	19.3%
2	terrorgram	Antisemitism/Jews	10037	16.7%
3	terrorgram	White Nationalism/Russia-Ukraine	7083	15.7%
4	white-supremacist	Antisemitism/Racism	35647	14.3%
5	white-supremacist	Tactical Training/Slurs	423	11.8%
6	terrorgram	Numbers/Nazi Numerology	218	10.9%
7	terrorgram	Non-English (Spanish/Italian)	1657	7.6%
8	white-supremacist	Non-English (Spanish/German)	430	7.2%
9	terrorgram	Saints/Martyrs	1722	6.5%
10	terrorgram	Paganism/Satanism	2033	6.5%

D3: Topic Feature Profiles

Topics are classified by their dominant DCM pattern relative to within-category medians.

```r
profiles <- topic_props %>%
  filter(topic >= 0) %>%
  group_by(category) %>%
  mutate(
    hi_oo = outgroup_othering  > median(outgroup_othering),
    hi_vl = violence_condoning > median(violence_condoning),
    hi_th = existential_threat > median(existential_threat),
    hi_if = identity_fusion    > median(identity_fusion)
  ) %>%
  ungroup() %>%
  mutate(
    n_high = hi_oo + hi_vl + hi_th + hi_if,
    profile = case_when(
      n_high == 4                 ~ "Full cocktail",
      hi_oo & hi_vl & hi_th      ~ "Othering + Violence + Threat",
      hi_oo & hi_vl              ~ "Othering + Violence",
      hi_oo & !hi_vl             ~ "Dehumanisation (othering only)",
      hi_vl & !hi_oo             ~ "Violence without othering",
      TRUE                       ~ "Low DCM"
    )
  ) %>%
  select(category, topic, label, n,
         identity_fusion, violence_condoning, existential_threat,
         outgroup_othering, composite_dcm, profile)

write_csv(profiles, file.path(OUTPUT_DIR, "D_topic_profiles.csv"))
```

```r
profiles %>%
  count(category, profile) %>%
  pivot_wider(names_from = category, values_from = n, values_fill = 0) %>%
  kable(caption = "Topic profile distribution by category")
```

Table 8: Topic profile distribution by category

profile	764	terrorgram	white-supremacist
Dehumanisation (othering only)	2	3	3
Full cocktail	4	5	3
Low DCM	8	7	7
Othering + Violence	2	0	2
Othering + Violence + Threat	1	1	1
Violence without othering	2	3	3

D4: Logistic Regression — Topic as Predictor of DCM Features

One model per feature per category (reference = Topic 0, the largest non-noise topic).

```r
logistic_topic <- map_dfr(FEATURES, function(feat) {
  map_dfr(unique(df_topics$category), function(cat) {
    sub <- df_topics %>%
      filter(category == cat, topic >= 0, !is.na(.data[[feat]])) %>%
      mutate(
        y = .data[[feat]],
        topic_f = relevel(factor(topic), ref = "0")
      )
    fit <- glm(y ~ topic_f, data = sub, family = binomial(link = "logit"))
    tidy(fit, conf.int = TRUE, exponentiate = TRUE) %>%
      filter(term != "(Intercept)") %>%
      mutate(
        feature  = FEATURE_LABELS[feat],
        category = cat,
        topic    = as.integer(str_extract(term, "\\d+"))
      ) %>%
      left_join(topic_labels %>% filter(category == cat),
                by = c("category", "topic")) %>%
      select(feature, category, topic, label,
             odds_ratio = estimate, ci_lo = conf.low, ci_hi = conf.high,
             p_value = p.value)
  })
})

write_csv(logistic_topic, file.path(OUTPUT_DIR, "D_topic_logistic.csv"))

logistic_topic %>%
  filter(p_value < 0.001) %>%
  arrange(feature, category, desc(odds_ratio)) %>%
  mutate(across(c(odds_ratio, ci_lo, ci_hi), ~ round(., 2)),
         p_value = formatC(p_value, format = "e", digits = 2)) %>%
  head(20) %>%
  kable(caption = "Top significant topic predictors of DCM features (p < .001, top 20 by OR)")
```

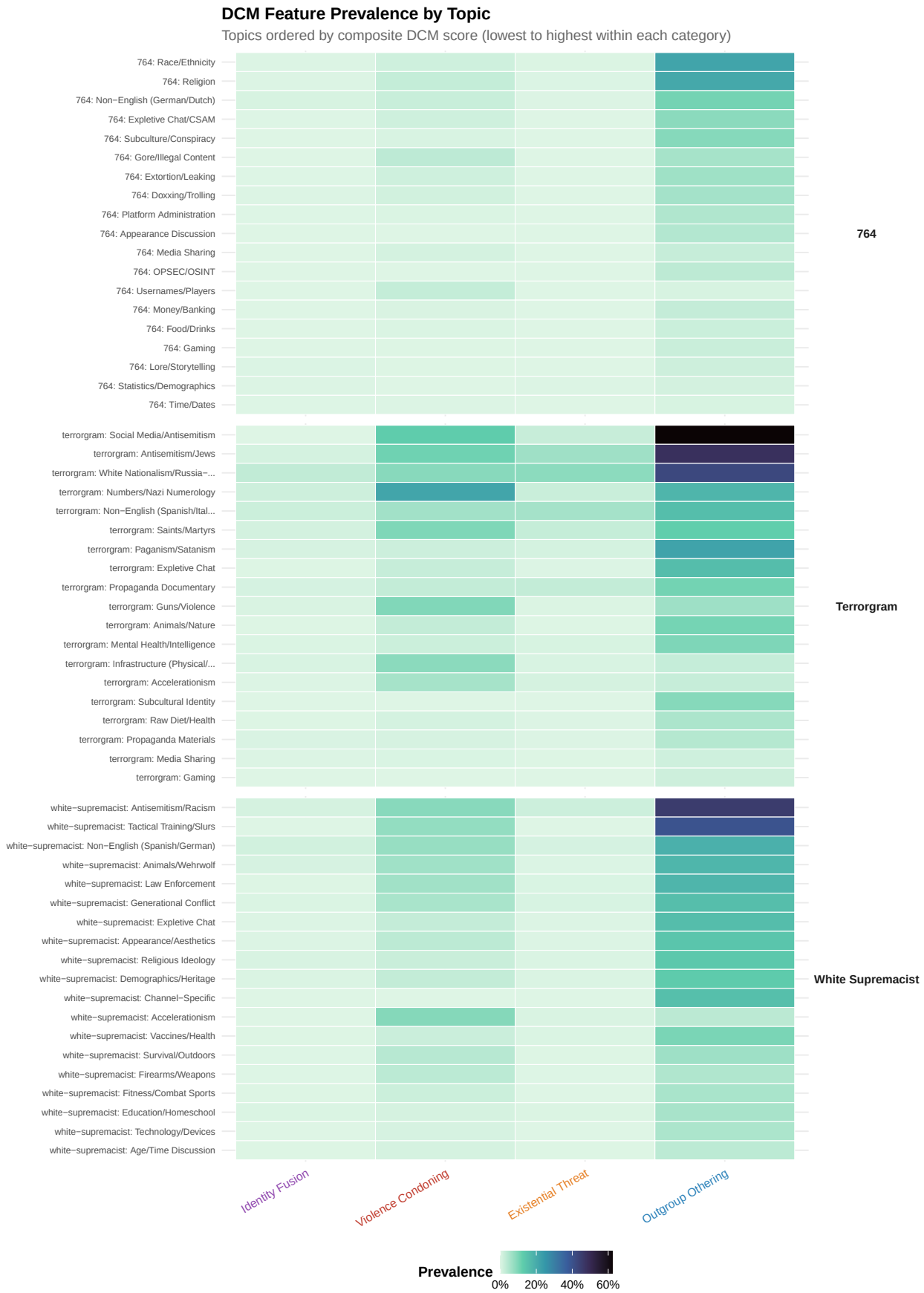
Table 9: Top significant topic predictors of DCM features ($p < .001$, top 20 by OR)

feature	category	topic	label	odds_ratio	ci_lo	ci_hi	p_value
Existential Threat	764	2	Race/Ethnicity	11.68	7.07	18.84	5.07e-23
Existential Threat	764	7	Religion	9.53	4.16	19.10	4.15e-09
Existential Threat	terrorgram	2	White Nationalism/Russia-Ukraine	42.53	32.75	56.35	3.51e-162
Existential Threat	terrorgram	1	Antisemitism/Jews	32.68	25.20	43.24	5.01e-142
Existential Threat	terrorgram	9	Non-English (Spanish/Italian)	29.46	21.32	41.09	3.82e-91
Existential Threat	terrorgram	11	Propaganda Documentary	13.65	9.23	20.10	8.23e-40
Existential Threat	terrorgram	10	Saints/Martyrs	12.87	8.70	18.94	3.82e-38
Existential Threat	terrorgram	13	Social Media/Antisemitism	11.91	6.35	20.86	1.82e-16
Existential Threat	terrorgram	16	Numbers/Nazi Numerology	10.77	3.73	24.60	4.55e-07
Existential Threat	terrorgram	7	Paganism/Satanism	5.25	3.17	8.41	2.09e-11
Existential Threat	terrorgram	8	Mental Health/Intelligence	3.53	1.92	6.07	1.44e-05
Existential Threat	white-supremacist	1	Antisemitism/Racism	7.99	6.72	9.56	2.45e-118
Existential Threat	white-supremacist	8	Generational Conflict	2.89	1.97	4.13	1.65e-08
Existential Threat	white-supremacist	4	Religious Ideology	1.85	1.31	2.55	3.07e-04
Existential Threat	white-supremacist	2	Firearms/Weapons	0.26	0.14	0.44	2.98e-06
Identity Fusion	764	16	Non-English (German/Dutch)	13.47	3.30	36.03	9.95e-06
Identity Fusion	764	2	Race/Ethnicity	4.47	2.69	7.08	1.14e-09
Identity Fusion	764	7	Religion	3.99	1.68	7.99	4.11e-04
Identity Fusion	terrorgram	2	White Nationalism/Russia-Ukraine	20.13	14.67	28.28	3.67e-72
Identity Fusion	terrorgram	9	Non-English (Spanish/Italian)	12.97	8.20	20.35	1.30e-28

Topic-Feature Heatmap

For each of the 19 topics per category, the proportion of messages flagged for each DCM feature. Topics are ordered bottom-to-top by composite DCM score within each category.

```r
topic_props_plot <- topic_props %>%
  filter(topic >= 0) %>%
  mutate(
    category = factor(category, levels = names(CATEGORY_LABELS)),
    label_short = str_trunc(label, 28)
  )

heatmap_data <- topic_props_plot %>%
  pivot_longer(
    cols = c(identity_fusion, violence_condoning, existential_threat, outgroup_othering),
    names_to = "feature", values_to = "proportion"
  ) %>%
  mutate(
    feature = recode(feature,
      identity_fusion = "Identity Fusion",
      violence_condoning = "Violence Condoning",
      existential_threat = "Existential Threat",
      outgroup_othering  = "Outgroup Othering"
    ),
    feature = factor(feature, levels = FEATURE_ORDER)
  )

topic_order <- topic_props_plot %>%
  arrange(category, composite_dcm) %>%
  mutate(topic_label = paste0(category, ": ", label_short)) %>%
  pull(topic_label)

heatmap_data <- heatmap_data %>%
  mutate(topic_label = factor(
    paste0(category, ": ", label_short),
    levels = topic_order
  ))

p_heatmap <- ggplot(heatmap_data, aes(x = feature, y = topic_label, fill = proportion)) +
  geom_tile(colour = "white", linewidth = 0.3) +
  facet_grid(category ~ ., scales = "free_y", space = "free_y",
             labeller = labeller(category = CATEGORY_LABELS)) +
  scale_fill_viridis_c(option = "mako", direction = -1, labels = percent_format(accuracy = 1),
                       name = "Prevalence") +
  labs(
    title = "DCM Feature Prevalence by Topic",
    subtitle = "Topics ordered by composite DCM score (lowest to highest within each category)",
    x = NULL, y = NULL
  ) +
  theme_pub() +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1, size = 9,
                               colour = FEATURE_COLORS[FEATURE_ORDER]),
    axis.text.y = element_text(size = 7),
    strip.text.y = element_text(angle = 0)
  )

p_heatmap
```

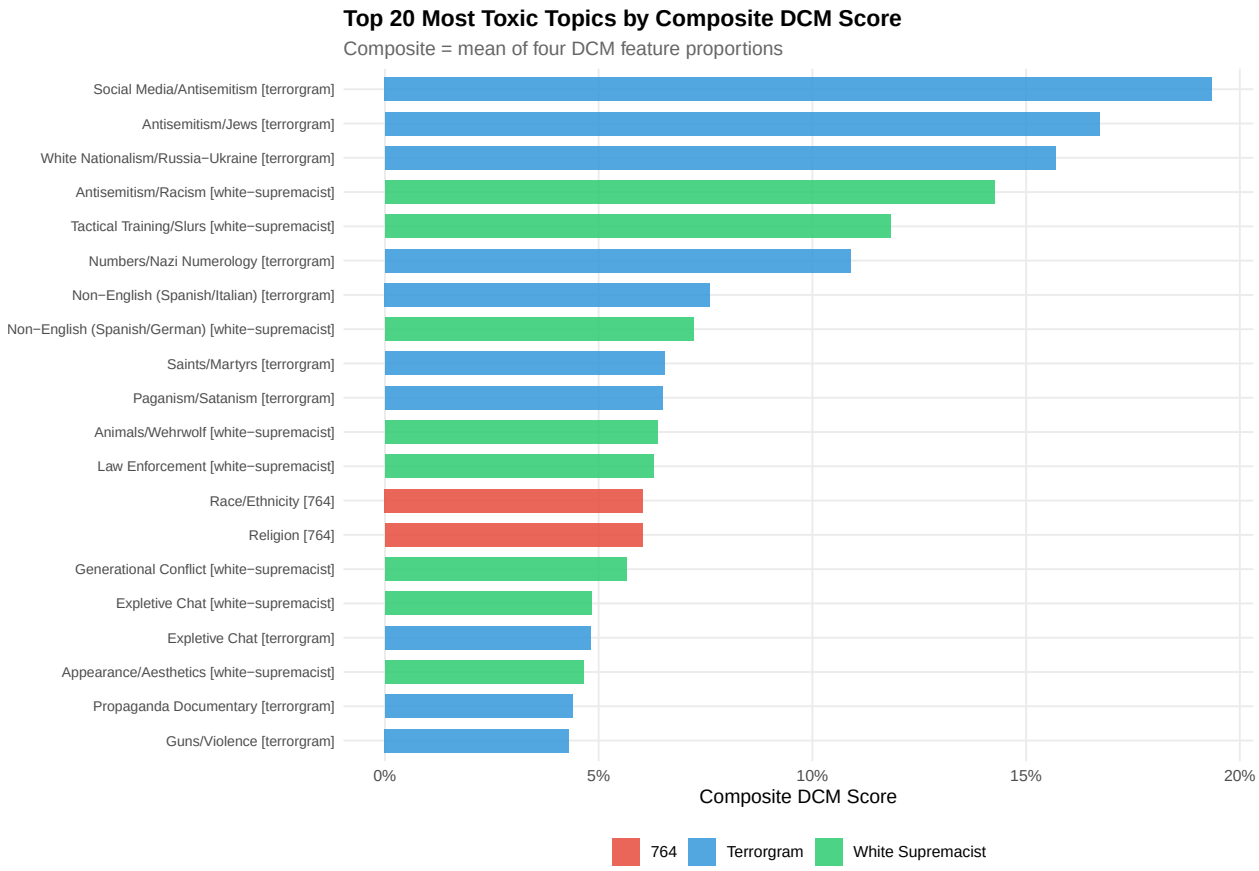

DCM Feature Prevalence by Topic
Topics ordered by composite DCM score (lowest to highest within each category)
764: Race/Ethnicity
764: Religion
764: Non−English (German/Dutch)
764: Expletive Chat/CSAM
764: Subculture/Conspiracy
764: Gore/Illegal Content
764: Extortion/Leaking
764: Doxxing/Trolling
764: Platform Administration
764: Appearance Discussion
764: Media Sharing
764: OPSEC/OSINT
764: Usernames/Players
764: Money/Banking
764: Food/Drinks
764: Gaming
764: Lore/Storytelling
764: Statistics/Demographics
764: Time/Dates
764
terrorgram: Social Media/Antisemitism
terrorgram: Antisemitism/Jews
terrorgram: White Nationalism/Russia−...
terrorgram: Numbers/Nazi Numerology
terrorgram: Non−English (Spanish/Ital...
terrorgram: Saints/Martyrs
terrorgram: Paganism/Satanism
terrorgram: Expletive Chat
terrorgram: Propaganda Documentary
terrorgram: Guns/Violence
terrorgram: Animals/Nature
terrorgram: Mental Health/Intelligence
terrorgram: Infrastructure (Physical/...
terrorgram: Accelerationism
terrorgram: Subcultural Identity
terrorgram: Raw Diet/Health
terrorgram: Propaganda Materials
terrorgram: Media Sharing
terrorgram: Gaming
Terrorgram
white−supremacist: Antisemitism/Racism
white−supremacist: Tactical Training/Slurs
white−supremacist: Non−English (Spanish/German)
white−supremacist: Animals/Wehrwolf
white−supremacist: Law Enforcement
white−supremacist: Generational Conflict
white−supremacist: Expletive Chat
white−supremacist: Appearance/Aesthetics
white−supremacist: Religious Ideology
white−supremacist: Demographics/Heritage
white−supremacist: Channel−Specific
white−supremacist: Accelerationism
white−supremacist: Vaccines/Health
white−supremacist: Survival/Outdoors
white−supremacist: Firearms/Weapons
white−supremacist: Fitness/Combat Sports
white−supremacist: Education/Homeschool
white−supremacist: Technology/Devices
white−supremacist: Age/Time Discussion
White Supremacist
Identity Fusion
Violence Condoning
Existential Threat
Outgroup Othering
Prevalence
0%
20%
40%
60%

```r
ggsave(file.path(FIGURES_DIR, "topic_heatmap.pdf"), p_heatmap, width = 10, height = 14)
```

Topic Toxicity Ranking (Top 20)

The 20 topics with the highest composite DCM score (mean of the four feature proportions), coloured by movement category.

```r
ranking_plot <- toxic_ranking %>%
  filter(rank <= 20) %>%
  mutate(
    category = factor(category, levels = names(CATEGORY_LABELS)),
    topic_label = paste0(label, " [", category, "]"),
    topic_label = fct_reorder(topic_label, composite_dcm)
  )

p_ranking <- ggplot(ranking_plot, aes(x = composite_dcm, y = topic_label, fill = category)) +
  geom_col(alpha = 0.85, width = 0.7) +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_x_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Top 20 Most Toxic Topics by Composite DCM Score",
    subtitle = "Composite = mean of four DCM feature proportions",
    x = "Composite DCM Score", y = NULL
  ) +
  theme_pub() +
  theme(axis.text.y = element_text(size = 8))

p_ranking
```




```
ggsave(file.path(FIGURES_DIR, "toxicity_ranking.pdf"), p_ranking, width = 10, height = 7)
```

Analysis E: Theme Prevalence Across Channel Types

Since topic models were fitted separately per category (yielding category-specific topics), direct cross-category comparison of individual topics is not possible. Analysis E addresses this by mapping each topic to one or more shared thematic categories (e.g., Antisemitism, White Supremacy, CSAM/Exploitation, Firearms/Weapons) defined in `topic_themes.csv`. It then tests whether the distribution of themes differs across the three channel types using a chi-square test of independence and reports standardized residuals to identify which themes are significantly over- or under-represented in each category.

Data Preparation

```
THEMES_PATH <- file.path(BASE_DIR, "topic_modelling/step5_cross_category/topic_themes.csv")

topic_themes <- read_csv(THEMES_PATH, col_types = cols(.default = "c")) %>%
  mutate(topic = as.integer(topic))

topic_themes_long <- topic_themes %>%
  separate_rows(themes, sep = ",\\s*") %>%
  mutate(themes = str_trim(themes))

df_themes <- df_topics %>%
  filter(topic >= 0) %>%
```

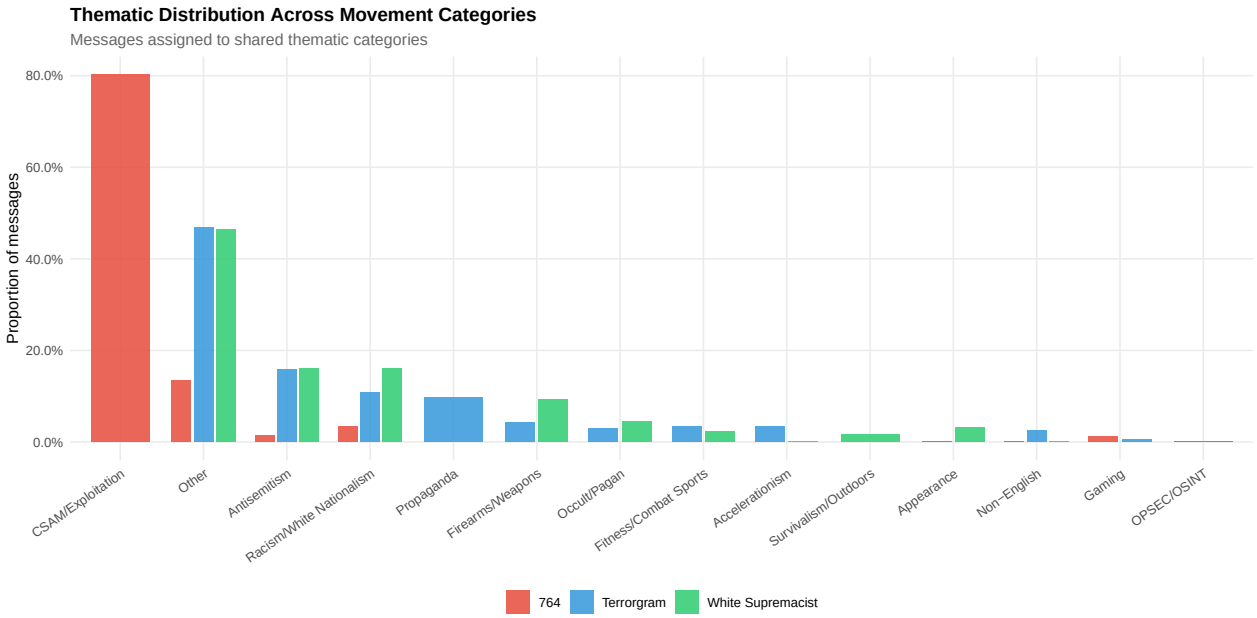
```r
  left_join(
    topic_themes_long %>% select(category, topic, theme = themes),
    by = c("category", "topic"),
    relationship = "many-to-many"
  ) %>%
  filter(!is.na(theme))
```

Theme join produced **500,903** message-theme rows.

E1: Theme Prevalence Proportions

```r
theme_prev <- df_themes %>%
  count(category, theme) %>%
  group_by(category) %>%
  mutate(prop = n / sum(n)) %>%
  ungroup()

write_csv(theme_prev, file.path(OUTPUT_DIR, "E_theme_prevalence.csv"))
```

```r
theme_prev_plot <- theme_prev %>%
  mutate(category = factor(category, levels = names(CATEGORY_LABELS)))

p_theme_prev <- ggplot(theme_prev_plot, aes(x = reorder(theme, -prop), y = prop, fill = category)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7, alpha = 0.85) +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_y_continuous(labels = percent_format(accuracy = 0.1)) +
  labs(
    title = "Thematic Distribution Across Movement Categories",
    subtitle = "Messages assigned to shared thematic categories",
    x = NULL, y = "Proportion of messages"
  ) +
  theme_pub() +
  theme(axis.text.x = element_text(angle = 35, hjust = 1))

p_theme_prev
```




```
ggsave(file.path(FIGURES_DIR, "theme_prevalence.pdf"), p_theme_prev, width = 12, height = 6)
```

E2: Chi-Square Test (Theme x Category)

```
theme_ct <- df_themes %>%
  count(category, theme) %>%
  pivot_wider(names_from = category, values_from = n, values_fill = 0) %>%
  column_to_rownames("theme") %>%
  as.matrix()

chisq_theme <- chisq.test(theme_ct)
n_total_theme <- sum(theme_ct)
k <- min(dim(theme_ct))
cramers_v_theme <- as.numeric(sqrt(chisq_theme$statistic / (n_total_theme * (k - 1))))

theme_chisq_result <- tibble(
  test       = "theme_x_category",
  chi_sq     = as.numeric(chisq_theme$statistic),
  df         = chisq_theme$parameter,
  p_value    = chisq_theme$p.value,
  cramers_v  = cramers_v_theme
)

write_csv(theme_chisq_result, file.path(OUTPUT_DIR, "E_theme_chisquare.csv"))

theme_chisq_result %>%
  mutate(
    chi_sq  = round(chi_sq, 1),
    p_value = formatC(p_value, format = "e", digits = 2),
    cramers_v = round(cramers_v, 3)
  ) %>%
  kable(caption = "Chi-square test of theme x category independence")
```

Table 10: Chi-square test of theme x category independence

test	chi_sq	df	p_value	cramers_v
theme_x_category	429676	26	0.00e+00	0.655

E3: Standardized Residuals

Values above +2 or below -2 indicate statistically significant over- or under-representation of a theme within a category.

```
std_resid <- as_tibble(as.data.frame.table(chisq_theme$stdres)) %>%
  rename(theme = Var1, category = Var2, std_residual = Freq)

write_csv(std_resid, file.path(OUTPUT_DIR, "E_theme_std_residuals.csv"))
```

```
std_resid_plot <- std_resid %>%
  mutate(category = factor(category, levels = names(CATEGORY_LABELS)))

p_resid <- ggplot(std_resid_plot, aes(x = category, y = theme, fill = std_residual)) +
  geom_tile(colour = "white", linewidth = 0.5) +
  geom_text(aes(label = sprintf("%.1f", std_residual)), size = 3.5) +
  scale_fill_gradient2(low = "#2166ac", mid = "white", high = "#b2182b",
                       midpoint = 0, name = "Std.\nResidual") +
  scale_x_discrete(labels = CATEGORY_LABELS) +
  labs(
    title = "Theme \u00d7 Category Standardized Residuals",
    subtitle = "Values > |2| indicate significant over/under-representation"
  ) +
  theme_pub() +
  theme(legend.position = "right")

p_resid
```

```
ggsave(file.path(FIGURES_DIR, "theme_residuals.pdf"), p_resid, width = 8, height = 6)
```

Analysis F: DCM-to-Topic (Reverse Direction)

Analysis F reverses the perspective of Analysis D: instead of asking "which topics carry the most DCM features?", it asks "for messages flagged with a given DCM feature, what topics and themes appear?" This reveals the thematic content that characterises radicalisation-relevant language. It reports within-category topic distributions for DCM+ vs DCM- messages, cross-category theme distributions restricted to DCM+ messages, and log-odds enrichment scores identifying which topics are most over-represented among DCM-positive messages.

F1: Topic Distribution for DCM+ vs DCM- Messages

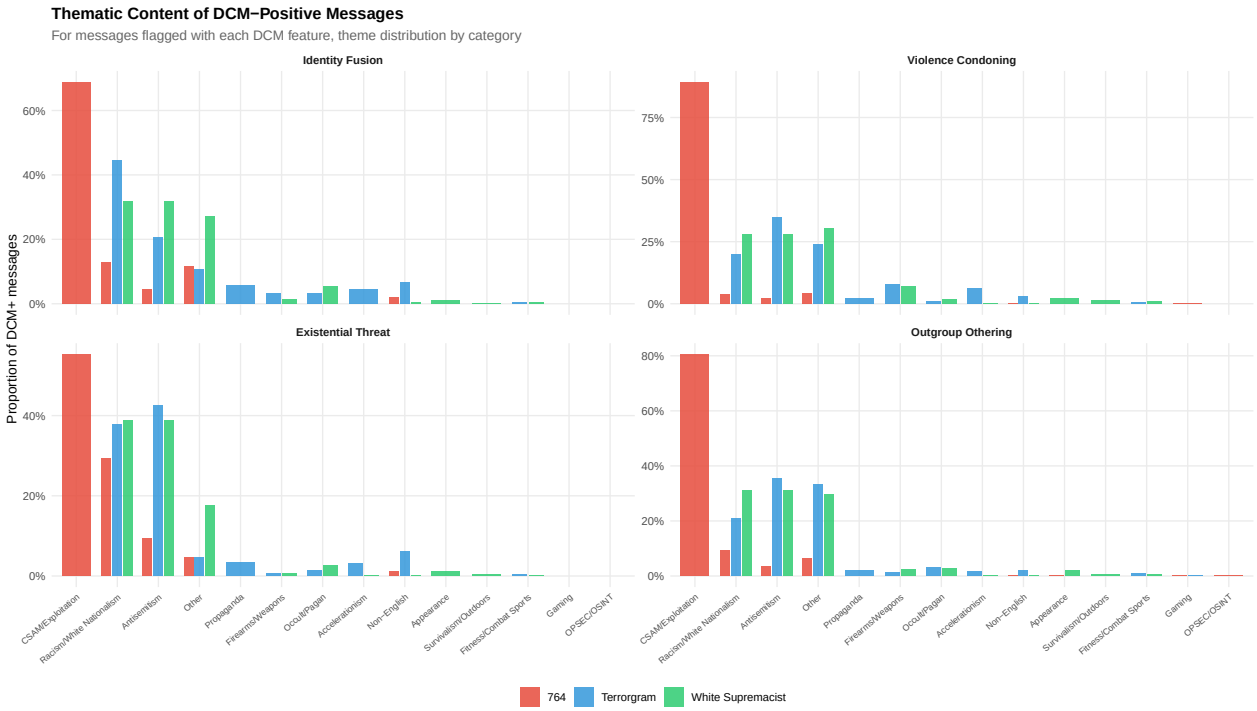
```
dcm_topic_dist <- map_dfr(FEATURES, function(feat) {
  map_dfr(unique(df_topics$category), function(cat) {
    sub <- df_topics %>%
      filter(category == cat, topic >= 0, !is.na(.data[[feat]]))
    sub %>%
      mutate(dcm_group = ifelse(.data[[feat]] == 1, "DCM+", "DCM-")) %>%
      count(dcm_group, topic, label) %>%
      group_by(dcm_group) %>%
      mutate(prop = n / sum(n)) %>%
```

```r
      ungroup() %>%
      mutate(feature = FEATURE_LABELS[feat], category = cat)
  })
})

write_csv(dcm_topic_dist, file.path(OUTPUT_DIR, "F_dcm_topic_distribution.csv"))
```

F2: Theme Distribution of DCM+ Messages

```r
dcm_theme_dist <- map_dfr(FEATURES, function(feat) {
  df_themes %>%
    filter(!is.na(.data[[feat]]), .data[[feat]] == 1) %>%
    count(category, theme) %>%
    group_by(category) %>%
    mutate(prop = n / sum(n)) %>%
    ungroup() %>%
    mutate(feature = FEATURE_LABELS[feat])
})

write_csv(dcm_theme_dist, file.path(OUTPUT_DIR, "F_dcm_theme_distribution.csv"))
```

```r
dcm_theme_plot <- dcm_theme_dist %>%
  mutate(
    category = factor(category, levels = names(CATEGORY_LABELS)),
    feature = factor(feature, levels = FEATURE_ORDER)
  )

p_dcm_theme <- ggplot(dcm_theme_plot, aes(x = reorder(theme, -prop), y = prop, fill = category)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7, alpha = 0.85) +
  facet_wrap(~feature, nrow = 2, scales = "free_y") +
  scale_fill_manual(values = CATEGORY_COLORS, labels = CATEGORY_LABELS, name = NULL) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Thematic Content of DCM-Positive Messages",
    subtitle = "For messages flagged with each DCM feature, theme distribution by category",
    x = NULL, y = "Proportion of DCM+ messages"
  ) +
  theme_pub() +
  theme(axis.text.x = element_text(angle = 40, hjust = 1, size = 7))

p_dcm_theme
```



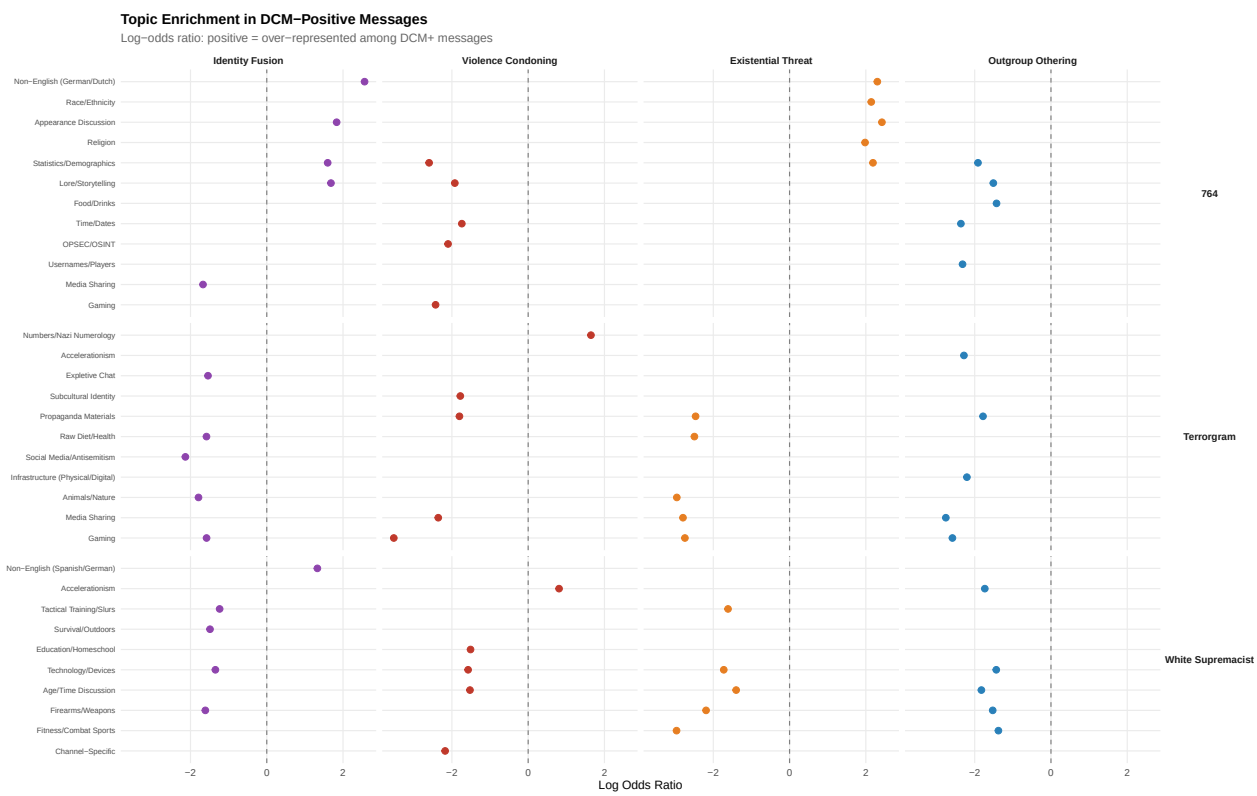

```
ggsave(file.path(FIGURES_DIR, "dcm_theme_dist.pdf"), p_dcm_theme, width = 14, height = 8)
```

F3: Log-Odds Topic Enrichment

Log-odds enrichment scores for topics in DCM+ vs DCM- messages, with Laplace smoothing. Positive values indicate topics that are over-represented among messages flagged with that DCM feature.

```
topic_enrichment <- map_dfr(FEATURES, function(feat) {
  map_dfr(unique(df_topics$category), function(cat) {
    sub <- df_topics %>%
      filter(category == cat, topic >= 0, !is.na(.data[[feat]]))
    n_pos <- sum(sub[[feat]] == 1)
    n_neg <- sum(sub[[feat]] == 0)
    if (n_pos == 0 || n_neg == 0) return(tibble())
    sub %>%
      group_by(topic, label) %>%
      summarise(
        n_dcm_pos = sum(.data[[feat]] == 1),
        n_dcm_neg = sum(.data[[feat]] == 0),
        .groups = "drop"
      ) %>%
      mutate(
        prop_pos = (n_dcm_pos + 0.5) / (n_pos + 1),
        prop_neg = (n_dcm_neg + 0.5) / (n_neg + 1),
        log_odds = log(prop_pos / prop_neg),
        feature  = FEATURE_LABELS[feat],
        category = cat
      )
  })
})

write_csv(topic_enrichment, file.path(OUTPUT_DIR, "F_topic_enrichment.csv"))
```

```r
enrichment_plot <- topic_enrichment %>%
  mutate(
    category = factor(category, levels = names(CATEGORY_LABELS)),
    feature = factor(feature, levels = FEATURE_ORDER)
  )

top_enriched <- enrichment_plot %>%
  group_by(feature, category) %>%
  slice_max(abs(log_odds), n = 5) %>%
  ungroup()

p_enrich <- ggplot(top_enriched, aes(x = log_odds, y = reorder(label, log_odds),
                                     colour = feature)) +
  geom_vline(xintercept = 0, linetype = "dashed", colour = "grey50") +
  geom_point(size = 2.5) +
  facet_grid(category ~ feature, scales = "free_y", space = "free_y",
             labeller = labeller(category = CATEGORY_LABELS)) +
  scale_colour_manual(values = FEATURE_COLORS, guide = "none") +
  labs(
    title = "Topic Enrichment in DCM-Positive Messages",
    subtitle = "Log-odds ratio: positive = over-represented among DCM+ messages",
    x = "Log Odds Ratio", y = NULL
  ) +
  theme_pub() +
  theme(axis.text.y = element_text(size = 7), strip.text.y = element_text(angle = 0))

p_enrich
```



```r
ggsave(file.path(FIGURES_DIR, "topic_enrichment.pdf"), p_enrich, width = 16, height = 10)
```

Analysis G: Shared vs. Movement-Unique Theme DCM Check

To rule out the possibility that convergent DCM patterns are simply driven by the small set of themes shared across movements (e.g., Antisemitism, Racism/WN), we classify each topic's theme as "shared" (the theme appears in two or more movements) or "unique" (the theme appears in only one movement) and compare DCM feature prevalence across the two groups.

G1: Theme Overlap Classification

```r
# Determine which themes appear in which movements
theme_by_movement <- topic_themes_long %>%
  distinct(category, theme = themes) %>%
  group_by(theme) %>%
  summarise(
    n_movements = n_distinct(category),
    movements   = paste(sort(unique(category)), collapse = ", "),
    .groups     = "drop"
  ) %>%
  mutate(overlap_status = ifelse(n_movements >= 2, "Shared", "Unique"))

theme_by_movement %>%
  arrange(desc(n_movements), theme) %>%
  kable(col.names = c("Theme", "N movements", "Movements", "Status"),
        caption = "Theme overlap classification")
```

Table 11: Theme overlap classification

Theme	N movements	Movements	Status
Antisemitism	3	764, terrorgram, white-supremacist	Shared
Non-English	3	764, terrorgram, white-supremacist	Shared
Other	3	764, terrorgram, white-supremacist	Shared
Racism/White Nationalism	3	764, terrorgram, white-supremacist	Shared
Accelerationism	2	terrorgram, white-supremacist	Shared
Appearance	2	764, white-supremacist	Shared
Firearms/Weapons	2	terrorgram, white-supremacist	Shared
Fitness/Combat Sports	2	terrorgram, white-supremacist	Shared
Gaming	2	764, terrorgram	Shared
Occult/Pagan	2	terrorgram, white-supremacist	Shared
CSAM/Exploitation	1	764	Unique
OPSEC/OSINT	1	764	Unique
Propaganda	1	terrorgram	Unique
Survivalism/Outdoors	1	white-supremacist	Unique

G2: Topic-Level Classification and DCM Comparison

```r
# Assign overlap status to each topic via its theme
topic_overlap <- topic_themes_long %>%
  rename(theme = themes) %>%
```

```r
  left_join(theme_by_movement %>% select(theme, overlap_status), by = "theme")

# For topics with multiple themes, classify as "Shared" if ANY theme is shared
topic_overlap_summary <- topic_overlap %>%
  group_by(category, topic, label) %>%
  summarise(
    overlap_status = ifelse(any(overlap_status == "Shared"), "Shared", "Unique"),
    .groups = "drop"
  )

# Join to message-level DCM data
df_overlap <- df_topics %>%
  filter(topic >= 0) %>%
  left_join(topic_overlap_summary %>% select(category, topic, overlap_status),
            by = c("category", "topic")) %>%
  filter(!is.na(overlap_status))

# Message-weighted DCM prevalence by overlap status within each movement
overlap_dcm <- df_overlap %>%
  group_by(category, overlap_status) %>%
  summarise(
    n_topics  = n_distinct(topic),
    n_messages = n(),
    identity_fusion    = mean(identityfusion_bin, na.rm = TRUE),
    violence_condoning = mean(violentlang_bin, na.rm = TRUE),
    existential_threat = mean(threat_bin, na.rm = TRUE),
    outgroup_othering  = mean(outgroup_othering_bin, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(composite_dcm = (identity_fusion + violence_condoning +
                            existential_threat + outgroup_othering) / 4)

write_csv(overlap_dcm, file.path(OUTPUT_DIR, "G_shared_unique_dcm.csv"))

overlap_dcm %>%
  mutate(across(c(identity_fusion, violence_condoning, existential_threat,
                  outgroup_othering, composite_dcm), ~ percent(., accuracy = 0.1))) %>%
  kable(col.names = c("Movement", "Overlap", "Topics", "Messages",
                      "Fusion", "Violence", "Threat", "Othering", "Composite"),
        caption = "DCM prevalence by theme overlap status (message-weighted)")
```

Table 12: DCM prevalence by theme overlap status (message-weighted)

Movement	Overlap	Topics	Messages	Fusion	Violence	Threat	Othering	Composite
764	Shared	13	41868	0.1%	0.9%	0.1%	8.2%	2.3%
764	Unique	6	170470	0.1%	1.8%	0.0%	8.3%	2.5%
terrorgram	Shared	16	60718	0.8%	5.5%	2.6%	23.5%	8.1%
terrorgram	Unique	3	6512	0.4%	1.2%	0.8%	4.5%	1.7%
white-supremacist	Shared	18	181867	0.4%	4.4%	0.6%	19.5%	6.2%
white-supremacist	Unique	1	3821	0.1%	4.2%	0.2%	6.7%	2.8%

G3: Statistical Test

We use a message-level logistic regression within each movement predicting each DCM feature from overlap status (shared vs. unique). A non-significant or small effect of overlap status would confirm that the DCM convergence is not driven by shared themes.

```
overlap_tests <- map_dfr(FEATURES, function(feat) {
  map_dfr(unique(df_overlap$category), function(cat) {
    sub <- df_overlap %>% filter(category == cat)

    # Check that both levels of overlap_status are present
    if (n_distinct(sub$overlap_status) < 2) {
      return(tibble(
        feature = FEATURE_LABELS[feat], category = cat,
        OR = NA_real_, ci_lower = NA_real_, ci_upper = NA_real_,
        p_value = NA_real_, note = "Only one overlap group present"
      ))
    }

    mod <- glm(as.formula(paste(feat, "~ overlap_status")),
               data = sub, family = binomial)
    tidy_mod <- tidy(mod, conf.int = TRUE, exponentiate = TRUE) %>%
      filter(term != "(Intercept)")

    tibble(
      feature  = FEATURE_LABELS[feat],
      category = cat,
      OR       = tidy_mod$estimate,
      ci_lower = tidy_mod$conf.low,
      ci_upper = tidy_mod$conf.high,
      p_value  = tidy_mod$p.value,
      note     = ""
    )
  })
})

write_csv(overlap_tests, file.path(OUTPUT_DIR, "G_overlap_logistic.csv"))

overlap_tests %>%
  mutate(
    OR = round(OR, 2),
    ci = paste0("[", round(ci_lower, 2), ", ", round(ci_upper, 2), "]"),
    p_value = ifelse(p_value < 0.001, "< .001", round(p_value, 3))
  ) %>%
  select(category, feature, OR, ci, p_value) %>%
  kable(col.names = c("Movement", "Feature", "OR (Unique vs Shared)", "95% CI", "p"),
        caption = "Logistic regression: overlap status predicting DCM features (reference = Shared)")
```

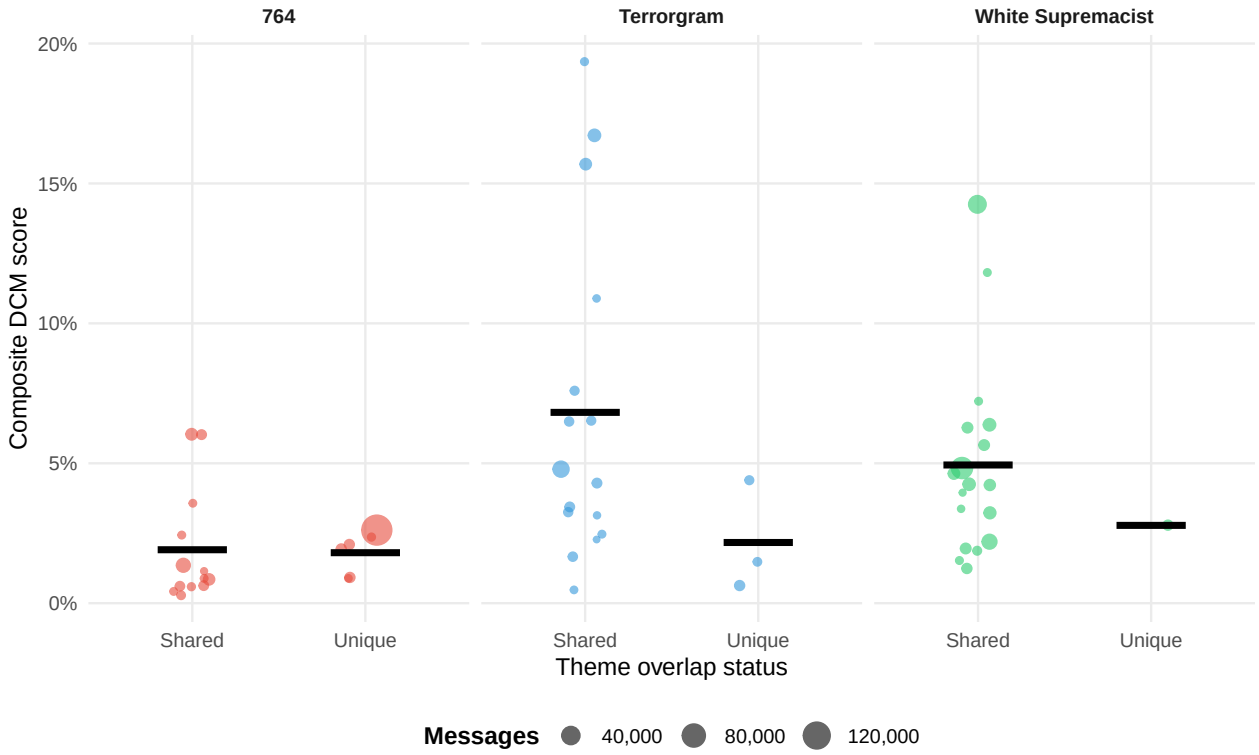
Table 13: Logistic regression: overlap status predicting DCM features (reference = Shared)

Movement	Feature	OR (Unique vs Shared)	95% CI	p
764	Identity Fusion	0.54	[0.39, 0.77]	< .001
terrorgram	Identity Fusion	0.56	[0.38, 0.8]	0.003

Movement	Feature	OR (Unique vs Shared)	95% CI	p
white-supremacist	Identity Fusion	0.19	[0.05, 0.5]	0.004
764	Violence Condoning	2.06	[1.85, 2.31]	< .001
terrorgram	Violence Condoning	0.20	[0.16, 0.25]	< .001
white-supremacist	Violence Condoning	0.94	[0.8, 1.1]	0.443
764	Existential Threat	0.30	[0.2, 0.47]	< .001
terrorgram	Existential Threat	0.32	[0.24, 0.41]	< .001
white-supremacist	Existential Threat	0.26	[0.1, 0.53]	0.001
764	Outgroup Othering	1.02	[0.98, 1.06]	0.261
terrorgram	Outgroup Othering	0.15	[0.14, 0.17]	< .001
white-supremacist	Outgroup Othering	0.30	[0.26, 0.34]	< .001

G4: Figure — Composite DCM by Overlap Status

```r
# Topic-level composite DCM with overlap status
topic_overlap_dcm <- topic_props %>%
  filter(topic >= 0) %>%
  left_join(topic_overlap_summary %>% select(category, topic, overlap_status),
            by = c("category", "topic")) %>%
  filter(!is.na(overlap_status))

p_overlap <- ggplot(topic_overlap_dcm,
                    aes(x = overlap_status, y = composite_dcm, colour = category)) +
  geom_jitter(aes(size = n), width = 0.15, alpha = 0.6) +
  stat_summary(fun = mean, geom = "crossbar", width = 0.4,
               linewidth = 0.6, colour = "black") +
  facet_wrap(~category, labeller = labeller(category = CATEGORY_LABELS)) +
  scale_colour_manual(values = CATEGORY_COLORS, guide = "none") +
  scale_size_continuous(name = "Messages", labels = comma) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    x = "Theme overlap status",
    y = "Composite DCM score"
  ) +
  theme_pub()

p_overlap
```




```
ggsave(file.path(FIGURES_DIR, "shared_unique_dcm.pdf"), p_overlap, width = 8, height = 5)
```

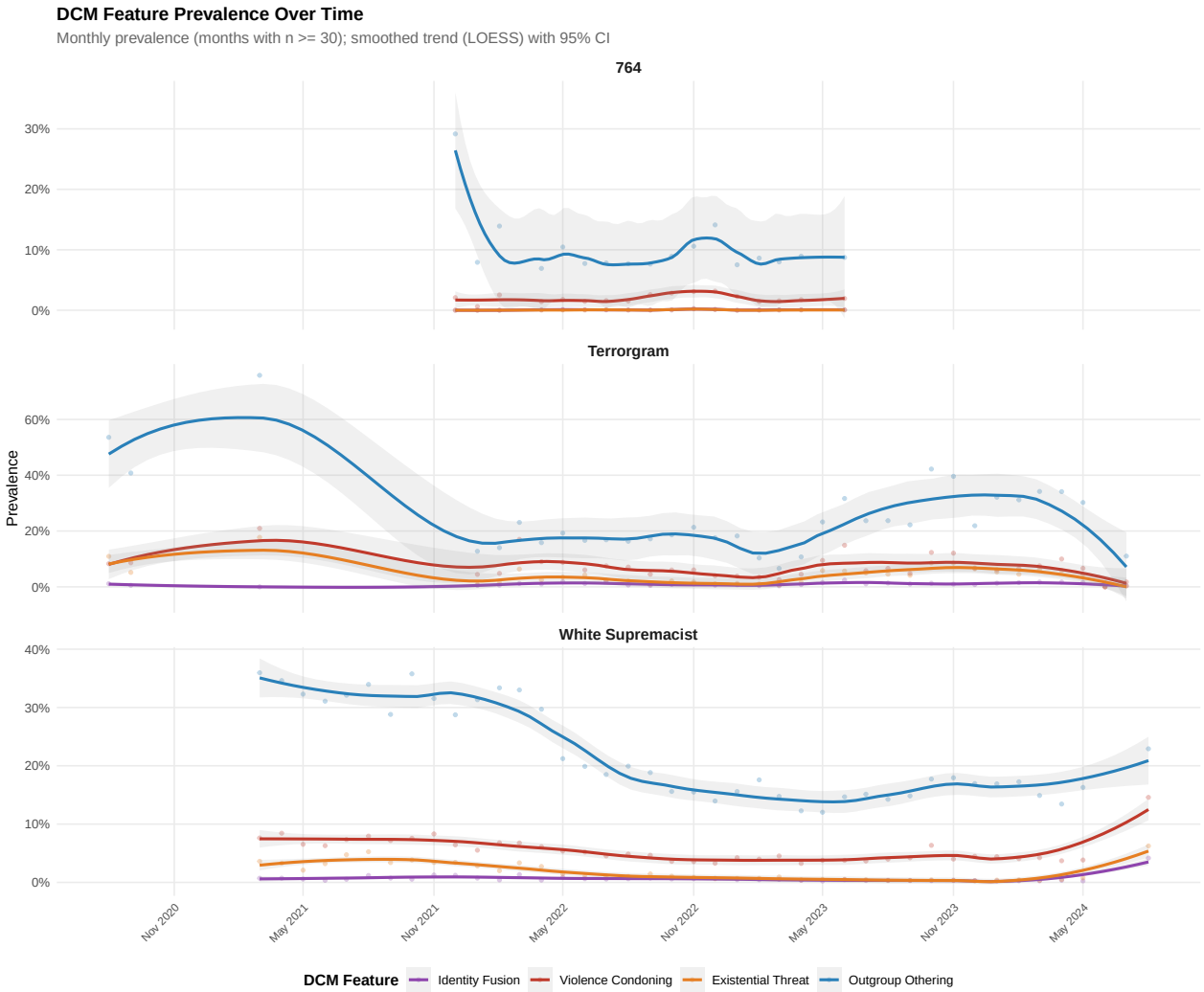
Temporal DCM Prevalence

Monthly DCM feature prevalence over time for each category, restricted to months with at least 30 messages. LOESS-smoothed trends with 95% confidence bands.

```
df_dated <- read_csv(DATA_PATH, col_types = cols(.default = "c")) %>%
  select(category, channel, date, all_of(FEATURES)) %>%
  mutate(
    across(all_of(FEATURES), as.integer),
    date = as.Date(date),
    year_month = floor_date(date, "month")
  ) %>%
  filter(!is.na(date))

temporal_dcm <- map_dfr(FEATURES, function(feat) {
  df_dated %>%
    filter(!is.na(.data[[feat]])) %>%
    group_by(category, year_month) %>%
    summarise(
      n = n(),
      k = sum(.data[[feat]]),
      prev = k / n,
      .groups = "drop"
    ) %>%
    mutate(feature = FEATURE_LABELS[feat])
})
```

```r
write_csv(temporal_dcm, file.path(OUTPUT_DIR, "temporal_dcm_prevalence.csv"))
```

```r
temporal_plot <- temporal_dcm %>%
  filter(n >= 30) %>%
  mutate(
    category = factor(category, levels = names(CATEGORY_LABELS)),
    feature = factor(feature, levels = FEATURE_ORDER),
    year_month = as.Date(year_month)
  )

p_temporal <- ggplot(temporal_plot, aes(x = year_month, y = prev, colour = feature)) +
  geom_point(size = 1, alpha = 0.3) +
  geom_smooth(method = "loess", se = TRUE, linewidth = 1, span = 0.4, alpha = 0.15) +
  facet_wrap(~category, ncol = 1, scales = "free_y",
             labeller = labeller(category = CATEGORY_LABELS)) +
  scale_colour_manual(values = FEATURE_COLORS, name = "DCM Feature") +
  scale_fill_manual(values = FEATURE_COLORS, guide = "none") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "6 months") +
  labs(
    title = "DCM Feature Prevalence Over Time",
    subtitle = "Monthly prevalence (months with n \u2265 30); smoothed trend (LOESS) with 95% CI",
    x = NULL, y = "Prevalence"
  ) +
  theme_pub() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1, size = 8),
    legend.position = "bottom",
    strip.text = element_text(size = 11)
  )

p_temporal
```




```
ggsave(file.path(FIGURES_DIR, "temporal_dcm.pdf"), p_temporal, width = 12, height = 10)
```

Transfer Entropy

Transfer entropy measures directed temporal dependencies between DCM features within each category. Using daily aggregated time series, multivariate transfer entropy tests whether past values of one DCM feature significantly improve predictions of another feature's future values, beyond what the target feature's own history predicts. Significant flows (FDR-corrected, $p < .05$) are shown as directed edges, with edge width proportional to the transfer entropy magnitude. Lag labels indicate the temporal offset (in days) at which the dependency is strongest.

Directed Temporal Dependencies

```
te <- read_csv(file.path(OUTPUT_DIR, "transfer_entropy.csv"), show_col_types = FALSE) %>%
  mutate(category = factor(category, levels = names(CATEGORY_LABELS)))

# Fixed node positions (diamond layout)
node_pos <- tibble(
  feature = factor(FEATURE_ORDER, levels = FEATURE_ORDER),
```

```r
  x = c(0, 1, 0, -1),
  y = c(1, 0, -1, 0)
)

NODE_RADIUS <- 0.35

edges <- te %>%
  left_join(node_pos, by = c("source" = "feature")) %>%
  rename(x_src = x, y_src = y) %>%
  left_join(node_pos, by = c("target" = "feature")) %>%
  rename(x_tgt = x, y_tgt = y) %>%
  mutate(
    dx = x_tgt - x_src, dy = y_tgt - y_src,
    len = sqrt(dx^2 + dy^2),
    x_start = x_src + NODE_RADIUS * dx / len,
    y_start = y_src + NODE_RADIUS * dy / len,
    x_end = x_tgt - NODE_RADIUS * dx / len,
    y_end = y_tgt - NODE_RADIUS * dy / len,
    lag_label = str_extract(lags, "\\d+"),
    perp_x = -dy / len * 0.14,
    perp_y =  dx / len * 0.14
  )

edges <- edges %>%
  group_by(category, target) %>%
  mutate(
    offset_sign = if_else(row_number() %% 2 == 0, -1, 1),
    lbl_x = (x_start + x_end) / 2 + perp_x * offset_sign,
    lbl_y = (y_start + y_end) / 2 + perp_y * offset_sign
  ) %>%
  ungroup()

nodes_all <- te %>%
  distinct(category) %>%
  crossing(node_pos)

p_te <- ggplot() +
  geom_segment(
    data = edges,
    aes(x = x_start, y = y_start, xend = x_end, yend = y_end, linewidth = te),
    arrow = arrow(length = unit(0.25, "cm"), type = "closed"),
    colour = "grey30", alpha = 0.85
  ) +
  scale_linewidth_continuous(range = c(0.4, 2), guide = "none") +
  geom_point(data = nodes_all, aes(x = x, y = y, fill = feature), size = 26,
             colour = "grey30", shape = 21, stroke = 0.6) +
  scale_fill_manual(values = FEATURE_COLORS, guide = "none") +
  geom_text(data = nodes_all, aes(x = x, y = y, label = str_wrap(feature, 10)),
            size = 3, lineheight = 0.85, fontface = "bold", colour = "white") +
  geom_label(
    data = edges,
    aes(x = lbl_x, y = lbl_y, label = paste0("lag ", lag_label)),
    size = 2.4, fill = "white", linewidth = 0.15, alpha = 0.92,
```

```
    label.padding = unit(0.12, "lines")
  ) +
  facet_wrap(~category, labeller = labeller(category = CATEGORY_LABELS), nrow = 1) +
  coord_fixed(xlim = c(-1.7, 1.7), ylim = c(-1.7, 1.7)) +
  labs(
    title = "Directed Temporal Dependencies Between DCM Features",
    subtitle = "Multivariate transfer entropy (lags 1\u20137 days); edges = significant flows (p < .05,
  ) +
  theme_pub() +
  theme(
    axis.text = element_blank(),
    axis.title = element_blank(),
    panel.grid = element_blank(),
    legend.position = "none"
  )

p_te
```

```
ggsave(file.path(FIGURES_DIR, "transfer_entropy.pdf"), p_te, width = 14, height = 5.5)
```